

SQL Cheat Sheet

27 tables · 190 concepts · Updated 2026-05-24 · cheatgrid.com

SQL (Structured Query Language) is the standard language for managing and manipulating relational databases, used across virtually all industries for data retrieval, analysis, and transformation. Whether querying a simple table or orchestrating complex multi-table joins with aggregations and window functions, SQL provides a **declarative syntax** where you specify *what* you want, not *how* to get it — the database engine handles optimization. Mastering SQL means understanding not just the syntax, but the **logical execution order** (FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY), which fundamentally differs from how queries are written and explains many common pitfalls.

CONTENTS

Basic Query Structure	2
DDL Statements	2
DML Statements	3
Join Types	4
Aggregate Functions	5
Window Functions — Ranking	5
Window Functions — Aggregate & Analytical	6
Window Frame Specification	7
Subqueries & CTEs	7
Set Operations	8
Advanced Grouping	8
PIVOT / UNPIVOT	9
Conditional Logic	9
String Functions	10
Date & Time Functions	11
Numeric Functions	12
NULL Handling	13
Pattern Matching	13
Data Type Conversion	14
Constraints & Keys	14
Indexes	15
Views	16
Transactions	16
Error Handling	17
JSON Functions	18
Query Execution Order	18
Advanced Ordering	19
References	19

TABLE 1

Basic Query Structure

Every SQL query is built from the same six core clauses, but they don't run in the order you write them — the engine processes **FROM** first and **SELECT** near the end, which is why aliases work in **ORDER BY** but not in **WHERE**. Knowing this logical processing order is the single most useful mental model for explaining why **HAVING** filters groups while **WHERE** filters rows, and why row-limiting clauses without **ORDER BY** return non-deterministic results.

Clause	Example	Description
SELECT	<code>SELECT name, salary FROM employees</code>	- Specifies which columns to return and computes their expressions - logically executed near the end (after FROM, WHERE, GROUP BY, HAVING), so its aliases are not visible to those clauses.
FROM	<code>FROM employees</code>	- Identifies the source tables, views, or subqueries and applies any JOINS - logically the first clause processed, so its tables and columns are visible to every later clause.
WHERE	<code>WHERE salary > 50000</code>	- Filters individual rows before grouping - cannot reference aggregate functions or SELECT-list aliases (SELECT hasn't run yet).
GROUP BY	<code>GROUP BY department</code>	- Collapses rows into one row per distinct group so aggregates like SUM and COUNT can be applied - every non-aggregated column in SELECT must also appear here.
HAVING	<code>HAVING COUNT(*) > 10</code>	- Filters groups after aggregation - the only clause where aggregate functions are valid filter conditions.
ORDER BY	<code>ORDER BY salary DESC</code>	- Sorts the final result set - runs after SELECT, so it can reference SELECT-list aliases; ASC is the default direction.
DISTINCT	<code>SELECT DISTINCT country FROM customers</code>	- Removes duplicate rows from the result, comparing the full tuple of selected columns - not per-column — every selected column counts toward uniqueness.
LIMIT / TOP / FETCH FIRST	<code>LIMIT 10</code> <code>TOP 10</code> <code>FETCH FIRST 10 ROWS ONLY</code>	- Caps the number of rows returned - syntax varies by dialect: LIMIT in PostgreSQL, MySQL, SQLite, Snowflake; TOP in SQL Server; FETCH FIRST is the ANSI standard supported by Oracle, DB2, Snowflake, and PostgreSQL.
OFFSET	<code>LIMIT 10 OFFSET 20</code>	Skips a number of rows before returning results, typically for pagination - without ORDER BY the skipped subset is undefined, so different pages can repeat or miss rows.
WINDOW clause	<code>SELECT name, RANK() OVER w</code> <code>FROM employees</code> <code>WINDOW w AS (PARTITION BY dept</code> <code>ORDER BY salary DESC)</code>	- Defines a named window specification that multiple window functions in the same query can reference via OVER w - removes duplication when several aggregates share one partition / order.

TABLE 2

DDL Statements

Data Definition Language (DDL) shapes the database itself — creating, altering, and dropping the tables, columns, and databases that hold your data. Unlike DML, several DDL statements behave very differently across engines: TRUNCATE is transactional in PostgreSQL and SQL Server but auto-commits in MySQL and Oracle, and rename syntax splits into ANSI **ALTER TABLE ... RENAME TO** vs SQL Server's **sp_rename**.

Statement	Example	Description
CREATE TABLE	<pre>CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(100) NOT NULL, email VARCHAR(255) UNIQUE)</pre>	Creates a new table with column definitions, data types, and constraints.
ALTER TABLE ADD COLUMN	<pre>ALTER TABLE employees ADD COLUMN phone VARCHAR(20)</pre>	• Adds a new column to an existing table • existing rows get NULL unless a DEFAULT is specified.
ALTER TABLE MODIFY / ALTER COLUMN	<pre>ALTER TABLE products ALTER COLUMN price DECIMAL(12,2)</pre>	• Changes a column's definition (data type, constraints) • syntax varies (MODIFY in MySQL, ALTER COLUMN in SQL Server / PostgreSQL).
ALTER TABLE DROP COLUMN	<pre>ALTER TABLE users DROP COLUMN legacy_flag</pre>	• Permanently removes a column and its data • in PostgreSQL it's metadata-only, so disk space isn't reclaimed until the table is rewritten
ALTER TABLE RENAME	<pre>ALTER TABLE customer RENAME TO customers</pre>	• Renames an existing table • syntax varies (RENAME TO in PostgreSQL / Oracle / MySQL, sp_rename in SQL Server).
DROP TABLE	<pre>DROP TABLE IF EXISTS temp_staging</pre>	• Permanently deletes a table, all its data, indexes, and triggers • IF EXISTS prevents an error if already absent; CASCADE removes dependent foreign-key constraints and views.
TRUNCATE TABLE	<pre>TRUNCATE TABLE audit_log</pre>	• Removes all rows while preserving table structure • much faster than DELETE ; does not fire row-level DELETE triggers; rollback works in PostgreSQL / SQL Server but auto-commits in MySQL / Oracle.
CREATE DATABASE	<pre>CREATE DATABASE analytics</pre>	Creates a new database on the server — a top-level container with its own schemas, users, and storage.

TABLE 3

DML Statements

Data Manipulation Language (DML) statements write to data rather than to the schema: **INSERT** adds rows, **UPDATE** modifies them, **DELETE** removes them, and **MERGE** does any combination of the three in one set-based statement. Every DML statement that modifies rows can be rolled back inside an explicit transaction — and every one of them silently affects every row in the table if you forget the **WHERE** clause.

Statement	Example	Description
INSERT INTO ... VALUES	<pre>INSERT INTO orders (customer_id, amount, status) VALUES (42, 299.99, 'pending')</pre>	• Inserts one or more rows into a table using literal values • column list is optional, but if omitted the values are matched positionally against the table's declared column order.
INSERT INTO ... SELECT	<pre>INSERT INTO archive SELECT * FROM orders WHERE order_date < '2025-01-01'</pre>	• Bulk-inserts rows from a query result; columns are paired positionally between SELECT and the target • standard pattern for copying, archiving, or staging data.
UPDATE ... SET	<pre>UPDATE employees SET salary = salary * 1.1 WHERE department = 'Engineering'</pre>	• Modifies existing rows that match the WHERE condition • without WHERE every row is updated — a classic production-disaster mistake.
UPDATE with JOIN	<pre>UPDATE e SET e.dept_name = d.name FROM employees e JOIN departments d ON e.dept_id = d.id</pre>	• Updates rows using data from a joined table • syntax differs across dialects: T-SQL uses UPDATE ... FROM ... JOIN , MySQL uses UPDATE t1 JOIN t2 ... SET , PostgreSQL uses UPDATE ... FROM with the join condition in WHERE.
DELETE FROM	<pre>DELETE FROM sessions WHERE last_active < NOW() - INTERVAL '30 days'</pre>	• Deletes matching rows and logs each row removal individually • fires row-level triggers and can be rolled back; without WHERE it empties the table (but leaves the schema intact).

Statement	Example	Description
MERGE (UPSERT)	<pre> MERGE target AS t USING source AS s ON t.id = s.id WHEN MATCHED THEN UPDATE SET t.val = s.val WHEN NOT MATCHED THEN INSERT (id, val) VALUES (s.id, s.val) WHEN NOT MATCHED BY SOURCE THEN DELETE; </pre>	• Combines INSERT, UPDATE, and DELETE in one statement based on a match condition (added to PostgreSQL in v15) • requires a trailing semicolon in SQL Server; under concurrency, WITH (HOLDLOCK) is needed to prevent the well-known UPSERT race condition.

TABLE 4

Join Types

Joins combine rows from two or more tables into a single result. Choosing the right join controls which rows survive when keys don't match on every side, and a misplaced filter or wrong join type is one of the most common sources of silently wrong query results.

Type	Example	Description
INNER JOIN	<pre> SELECT * FROM orders o INNER JOIN customers c ON o.customer_id = c.id </pre>	• Returns only rows with matching values in both tables • the default and most common join • rows with NULL keys on either side are excluded because NULL never equals NULL.
LEFT JOIN (LEFT OUTER JOIN)	<pre> SELECT * FROM customers c LEFT JOIN orders o ON c.id = o.customer_id </pre>	• Returns all rows from the left table plus matching rows from the right • non-matching right rows produce NULL • a filter on a right-table column in WHERE silently converts the join back to an INNER JOIN — put such filters in the ON clause instead.
RIGHT JOIN (RIGHT OUTER JOIN)	<pre> SELECT * FROM orders o RIGHT JOIN customers c ON o.customer_id = c.id </pre>	• Returns all rows from the right table plus matching rows from the left • mirror of LEFT JOIN; rarely used because flipping the table order to a LEFT JOIN is more readable and more portable.
FULL OUTER JOIN	<pre> SELECT * FROM customers c FULL OUTER JOIN orders o ON c.id = o.customer_id </pre>	• Returns all rows from both tables, with NULLs for the side that didn't match • not supported in MySQL — emulate by UNION ing a LEFT JOIN with a RIGHT JOIN.
CROSS JOIN	<pre> SELECT * FROM colors CROSS JOIN sizes </pre>	• Produces a Cartesian product — every row from the first table paired with every row from the second ($N \times M$ rows) • takes no ON condition • the old comma syntax FROM a, b produces the same Cartesian product.
SELF JOIN	<pre> SELECT e.name, m.name AS manager FROM employees e JOIN employees m ON e.manager_id = m.id </pre>	• Joins a table to itself using two different aliases • useful for hierarchical data (employee–manager) or comparing rows within the same table • aliases are mandatory — without them the column references are ambiguous.
LATERAL JOIN (CROSS JOIN LATERAL)	<pre> SELECT d.name, e.name, e.salary FROM departments d CROSS JOIN LATERAL (SELECT name, salary FROM employees WHERE department_id = d.id ORDER BY salary DESC LIMIT 3) e </pre>	• Allows the right-hand subquery to reference columns from tables earlier in the FROM clause, evaluating it once per outer row (like a correlated subquery in the FROM list) • enables top-N-per-group and per-row table-function calls that a plain JOIN cannot express • pair with LEFT JOIN LATERAL ... ON true to keep outer rows that produce no inner rows.
CROSS APPLY / OUTER APPLY	<pre> SELECT d.name, e.name FROM departments d CROSS APPLY (SELECT TOP 3 name FROM employees WHERE department_id = d.id ORDER BY salary DESC) e </pre>	• SQL Server's equivalent of LATERAL: evaluates the right-hand table expression once per left-hand row • CROSS APPLY drops left rows whose right expression returns no rows (like INNER JOIN) • OUTER APPLY keeps them with NULLs on the right (like LEFT JOIN).

Type	Example	Description
NATURAL JOIN	<code>SELECT * FROM employees NATURAL JOIN departments</code>	• Automatically joins on every column with the same name in both tables and collapses those columns into one • risky in production — adding a same-named column to either table later silently changes the join condition • prefer explicit <code>ON</code> or <code>USING (col)</code> .

TABLE 5

Aggregate Functions

Aggregate functions collapse many rows into a single summary value over a group, and almost all of them silently ignore NULL inputs. Two traps catch even experienced users: `COUNT(*)` counts every row while `COUNT(col)` skips NULLs in `col` , and `SUM` / `AVG` / `MIN` / `MAX` over an all-NULL or empty group return NULL — never zero.

Function	Example	Description
COUNT()	<code>SELECT COUNT(*) FROM orders</code> <code>SELECT COUNT(DISTINCT customer_id) FROM orders</code>	• <code>COUNT(*)</code> counts all rows including NULLs • <code>COUNT(col)</code> skips NULLs; <code>DISTINCT</code> counts unique values.
SUM()	<code>SELECT SUM(amount) FROM payments</code>	• Totals all non-NULL values in a column • returns NULL if all values are NULL.
AVG()	<code>SELECT AVG(salary) FROM employees WHERE dept = 'Sales'</code>	• Arithmetic mean of non-NULL values • NULLs are excluded, not treated as zero.
MAX()	<code>SELECT MAX(order_date) FROM orders</code>	• Returns the largest value in a column • works on dates and strings too.
MIN()	<code>SELECT MIN(price) FROM products WHERE active = 1</code>	Returns the smallest value in a column.
GROUP_CONCAT / STRING_AGG	<code>STRING_AGG(name, ', ') WITHIN GROUP (ORDER BY name)</code>	• Concatenates string values from a group into a single delimited string • SQL Server/PostgreSQL use <code>STRING_AGG</code> • MySQL uses <code>GROUP_CONCAT</code>.
VARIANCE / VAR_SAMP	<code>SELECT VAR(salary) FROM employees</code> <code>SELECT VARIANCE(salary) FROM employees</code>	• Statistical variance of the values in a group • <code>VAR/VAR_SAMP</code> (sample), <code>VARP/VAR_POP</code> (population) • name varies by database.
STDDEV / STDEV	<code>SELECT STDEV(salary) FROM employees</code>	• Standard deviation of values • <code>STDEV/STDDEV_SAMP</code> (sample), <code>STDEVP/STDDEV_POP</code> (population) • name varies by database.

TABLE 6

Window Functions — Ranking

The ranking family assigns a position number to each row inside an ordered partition. The differences between them are entirely about how ties and gaps are handled, so picking the wrong one quietly changes results. Every ranking function requires `ORDER BY` inside its `OVER` clause; `PARTITION BY` is optional and restarts the numbering for each group.

Function	Example	Description
ROW_NUMBER()	<code>ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC)</code>	• Assigns a unique sequential integer per row within a partition • ties receive different numbers (broken by <code>ORDER BY</code> or arbitrarily).
RANK()	<code>RANK() OVER (ORDER BY score DESC)</code>	• Assigns rank with gaps after ties — two rows tied at rank 2 are both rank 2; next row is rank 4 • competition-style ranking.
DENSE_RANK()	<code>DENSE_RANK() OVER (ORDER BY score DESC)</code>	• Like <code>RANK</code> but no gaps — two rows tied at rank 2 are both rank 2 • next row is rank 3.

Function	Example	Description
NTILE()	<code>NTILE(4) OVER (ORDER BY salary)</code>	- Divides the ordered partition into N buckets as equal as possible - if rows don't divide evenly the earliest buckets get the extra rows.
PERCENT_RANK()	<code>PERCENT_RANK() OVER (ORDER BY salary)</code>	- Relative rank $(rank - 1) / (total\ rows - 1)$, range 0 to 1 inclusive - the first row is always 0.0.
CUME_DIST()	<code>CUME_DIST() OVER (ORDER BY salary)</code>	- Cumulative distribution — rows with value $\leq$ current row, divided by total rows - range 1/N to 1 - tied rows share the same value.

TABLE 7

Window Functions — Aggregate & Analytical

Window functions compute a value for each row using a *window* of related rows, without collapsing the result the way `GROUP BY` does. The window is shaped by three parts of the `OVER` clause — `PARTITION BY` (which rows belong together), `ORDER BY` (their sequence), and the optional frame (`ROWS / RANGE` — which of those rows are visible from the current one). Most window-function surprises come from the frame: adding `ORDER BY` without an explicit frame defaults to `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, which silently turns whole-partition aggregates into running ones and makes `LAST_VALUE / NTH_VALUE` return the wrong row.

Function	Example	Description
SUM() OVER()	<code>SUM(amount) OVER (PARTITION BY dept ORDER BY date)</code>	Running total within a partition when <code>ORDER BY</code> is present - without <code>ORDER BY</code>, returns the partition total beside every row.
AVG() OVER()	<code>AVG(salary) OVER (PARTITION BY department_id)</code>	Partition mean on every row when no <code>ORDER BY</code> is given - adding <code>ORDER BY</code> turns it into a running average.
COUNT() OVER()	<code>COUNT(*) OVER (PARTITION BY customer_id)</code>	- Returns the number of rows in the partition beside every detail row - useful for showing a per-group total without <code>GROUP BY</code>.
LAG()	<code>LAG(close_price, 1, 0) OVER (ORDER BY date)</code>	- Returns the value offset rows before the current row in the window - the first row returns <code>NULL</code> unless a default is given as the third argument.
LEAD()	<code>LEAD(close_price, 1, 0) OVER (ORDER BY date)</code>	- Returns the value offset rows after the current row in the window - the last row returns <code>NULL</code> unless a default is given as the third argument.
FIRST_VALUE()	<code>FIRST_VALUE(salary) OVER (PARTITION BY dept ORDER BY hire_date)</code>	- Returns the value at the first row of the window frame - the default frame starts at the partition's first row, so the result is usually what you expect.
LAST_VALUE()	<code>LAST_VALUE(salary) OVER (PARTITION BY dept ORDER BY hire_date ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)</code>	- Returns the value at the last row of the window frame - the default frame stops at the current row, so an explicit <code>ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING</code> is needed for the true partition last value.
NTH_VALUE()	<code>NTH_VALUE(salary, 2) OVER (PARTITION BY dept ORDER BY salary DESC ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)</code>	- Returns the value at the nth row of the window frame (counting from 1) - returns <code>NULL</code> if <code>n</code> exceeds the frame, so expand the frame to <code>UNBOUNDED FOLLOWING</code> when you need access to rows past the current position.

TABLE 8

Window Frame Specification

The frame clause narrows a window function's view to a subset of the partition relative to the current row, controlling exactly which rows feed running totals, moving averages, and lookups like `LAST_VALUE()`. The most important and most surprising rule: when `ORDER BY` is present but no frame is specified, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` (peer-based, not row-based) — which silently fuses tied rows into a single running-sum step.

Clause	Example	Description
ROWS BETWEEN ... AND ...	<code>SUM(amount) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)</code>	- Frame defined by physical row offsets — counts rows regardless of their <code>ORDER BY</code> values - preferred for fixed-size moving calculations like 7-day windows.
RANGE BETWEEN ... AND ...	<code>SUM(amount) OVER (ORDER BY date RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)</code>	- Frame defined by logical value ranges — all rows that are <code>ORDER BY</code> peers of the current row are included together - this is the default frame when <code>ORDER BY</code> is present and no frame clause is given.
UNBOUNDED PRECEDING	<code>ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW</code>	Frame starts at the first row of the current partition — used for cumulative aggregations from the partition start.
UNBOUNDED FOLLOWING	<code>ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING</code>	Frame extends to the last row of the current partition — required for <code>LAST_VALUE()</code> and <code>NTH_VALUE()</code> to see all subsequent rows in standard SQL.
N PRECEDING / N FOLLOWING	<code>ROWS BETWEEN 3 PRECEDING AND 1 FOLLOWING</code>	- Frame spans N rows before to M rows after the current row - in <code>ROWS</code> mode the offset must be a non-negative integer; in <code>RANGE</code> mode it requires exactly one numeric or date <code>ORDER BY</code> column.
CURRENT ROW	<code>ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW</code>	- In <code>ROWS</code> mode means just the current row; in <code>RANGE/GROUPS</code> mode it spans all peer rows with the same <code>ORDER BY</code> value - this peer behavior is what produces the running-total surprise on duplicate ordering keys.

TABLE 9

Subqueries & CTEs

Subqueries nest one query inside another to compute filters, derived tables, or single values, while Common Table Expressions (CTEs) name those intermediate results to break a complex query into readable, sometimes recursive, steps. Knowing when each one short-circuits, gets correlated per row, or is materialized once is the difference between a query that scales and one that crawls.

Technique	Example	Description
Subquery (WHERE)	<code>SELECT * FROM employees WHERE dept_id IN (SELECT id FROM departments WHERE budget > 100000)</code>	- Embeds a query inside another query for filtering - the inner query runs once (non-correlated) or per row (correlated).
Correlated Subquery	<code>SELECT * FROM employees e1 WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e2.dept = e1.dept)</code>	- References an outer query's columns inside the subquery - logically executes once per outer row, though modern optimizers often decorrelate it into a join.
Subquery (FROM — Derived Table)	<code>SELECT dept, avg_sal FROM (SELECT dept, AVG(salary) AS avg_sal FROM employees GROUP BY dept) t</code>	- A subquery in the FROM clause - must have a table alias in most dialects - computed once and referenced like a table.
EXISTS	<code>SELECT name FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id)</code>	- Tests for row existence and stops at the first match <code>NOT EXISTS</code> is also the safe replacement for <code>NOT IN</code> when the subquery may contain NULLs.

Technique	Example	Description
CTE (Common Table Expression)	<pre>WITH ranked AS (SELECT *, RANK() OVER (ORDER BY salary DESC) AS rk FROM employees) SELECT * FROM ranked WHERE rk <= 10</pre>	- Named temporary result set introduced with <code>WITH</code> and scoped to a single statement - PostgreSQL 12+ may inline (fold) a CTE unless you write <code>AS MATERIALIZED</code>.
Recursive CTE	<pre>WITH RECURSIVE org AS (SELECT id, name, manager_id FROM employees WHERE manager_id IS NULL UNION ALL SELECT e.id, e.name, e.manager_id FROM employees e JOIN org o ON e.manager_id = o.id) SELECT * FROM org</pre>	- CTE that references itself to traverse hierarchical or graph data (org charts, bills of materials) - requires an anchor member, <code>UNION ALL</code> (or <code>UNION</code>) with a recursive member, and a condition that eventually returns no rows.
Scalar Subquery	<pre>SELECT name, (SELECT MAX(salary) FROM employees) AS max_sal FROM employees</pre>	- A subquery used as a single value (one row, one column) in <code>SELECT</code>, <code>WHERE</code>, etc. - returns <code>NULL</code> if zero rows and raises a cardinality error if it returns more than one.

TABLE 10

Set Operations

Set operators combine the rows of two or more `SELECT` queries that return the same number of columns with compatible data types. The result uses the column names from the first query, any `ORDER BY` must come at the very end of the combined statement, and `NULL`s are treated as equal for the purposes of duplicate elimination — a deliberate exception to SQL's usual three-valued logic.

Operation	Example	Description
UNION	<pre>SELECT city FROM customers UNION SELECT city FROM suppliers</pre>	- Combines result sets and removes duplicates (same logic as <code>DISTINCT</code>) - slower than <code>UNION ALL</code> because of the extra deduplication step.
UNION ALL	<pre>SELECT product_id FROM orders_2024 UNION ALL SELECT product_id FROM orders_2025</pre>	- Combines result sets including all duplicates - faster than <code>UNION</code> — the canonical guidance is to default to <code>UNION ALL</code> and only use <code>UNION</code> when you actually need duplicates removed.
INTERSECT	<pre>SELECT customer_id FROM orders_jan INTERSECT SELECT customer_id FROM orders_feb</pre>	- Returns rows that appear in both result sets - result is deduplicated; some dialects (e.g. PostgreSQL) also support <code>INTERSECT ALL</code> to keep duplicate matches.
EXCEPT (MINUS)	<pre>SELECT customer_id FROM customers EXCEPT SELECT customer_id FROM orders</pre>	- Returns rows in the first set that do not appear in the second <code>EXCEPT</code> is the ANSI name (PostgreSQL, SQL Server, MySQL 8.0.31+); Oracle uses <code>MINUS</code> - result is deduplicated.

TABLE 11

Advanced Grouping

GROUP BY extensions add multi-level subtotals and grand totals in a single pass over the data, replacing stacked `UNION ALL` queries with concise, optimizer-friendly syntax. ROLLUP gives strict hierarchical subtotals, CUBE gives every combination, and GROUPING SETS lets you cherry-pick exactly the sets you want; `GROUPING()` and `GROUPING_ID()` then let you tell super-aggregate rows apart from rows that happen to contain real NULLs.

Extension	Example	Description
ROLLUP	<pre>SELECT year, quarter, SUM(sales) FROM revenue GROUP BY ROLLUP(year, quarter)</pre>	- Generates hierarchical subtotals — produces N+1 grouping sets for N columns (each prefix of the column list plus the grand total) - e.g., totals by (year, quarter), (year), and grand total in one query.
CUBE	<pre>SELECT dept, year, SUM(revenue) FROM sales GROUP BY CUBE(dept, year)</pre>	- Generates every subset of the listed columns — 2^N grouping sets for N columns (the full power set) - includes every cross-tabulation plus the grand total.

Extension	Example	Description
GROUPING SETS	GROUP BY GROUPING SETS ((dept, year), (dept), (year), ())	• Explicit control over which grouping combinations to compute — no implicit hierarchy, no full crossing • avoids paying for CUBE combinations you don't need.
GROUPING()	SELECT GROUPING(dept), dept, SUM(sal) FROM employees GROUP BY ROLLUP(dept)	• Returns 1 for super-aggregate rows (where the column was rolled up to NULL) and 0 for normal rows • the only reliable way to distinguish ROLLUP/CUBE nulls from actual NULL values in the data.
GROUPING_ID()	SELECT GROUPING_ID(dept, year), dept, year, SUM(sal) FROM employees GROUP BY ROLLUP(dept, year)	• Returns an integer bitmap where each bit is the <code>GROUPING()</code> value for that column — identifies the row's grouping level in a single value • cleaner than multiple <code>GROUPING()</code> calls when you need to label, sort, or filter by level.

TABLE 12

PIVOT / UNPIVOT

PIVOT and UNPIVOT reshape result sets between long (one row per value) and wide (one column per category) layouts, which is the core operation behind most cross-tab reports. They are T-SQL extensions also supported in Oracle and Snowflake (with dialect-specific syntax), but PostgreSQL and MySQL have no native `PIVOT` — for those engines, and for any portable code, use conditional aggregation with `CASE` inside aggregate functions.

Operator	Example	Description
PIVOT	SELECT dept, [2023], [2024], [2025] FROM sales_data PIVOT (SUM(amount) FOR year IN ([2023],[2024],[2025])) AS pvt	• Rotates row values into columns — aggregates a measure and spreads distinct values from one column into separate columns • Aggregate function is always required, even when each cell maps to one row; values not in the <code>IN</code> list are silently dropped.
UNPIVOT	SELECT dept, year, amount FROM wide_table UNPIVOT (amount FOR year IN ([2023],[2024],[2025])) AS unpvt	• Reverses a pivot — turns column headers back into row values to normalise wide tables • In SQL Server, <code>NULL</code>-valued input cells produce no output row (Oracle exposes <code>INCLUDE NULLS</code> / <code>EXCLUDE NULLS</code>; T-SQL defaults to drop).
Dynamic PIVOT (Conditional Aggregation)	SELECT dept, SUM(CASE WHEN year=2024 THEN sales END) AS [2024], SUM(CASE WHEN year=2025 THEN sales END) AS [2025] FROM sales GROUP BY dept	• ANSI-standard, portable alternative to <code>PIVOT</code> using <code>CASE</code> inside aggregate functions — runs unchanged on SQL Server, PostgreSQL, MySQL, Oracle, SQLite • Also the standard workaround when pivoted columns are determined at runtime, paired with dynamic SQL to build the column list.

TABLE 13

Conditional Logic

Conditional expressions let a query branch row-by-row inside `SELECT`, `WHERE`, `ORDER BY`, and `UPDATE` clauses without leaving SQL. `CASE` is the portable, ANSI-standard primitive; vendors layer terser shorthands like `IIF`, `CHOOSE`, and Oracle's `DECODE` on top — each with its own NULL-handling quirks worth knowing.

Expression	Example	Description
CASE WHEN ... THEN ... END	CASE WHEN salary > 100000 THEN 'Senior' WHEN salary > 60000 THEN 'Mid' ELSE 'Junior' END	• SQL's primary conditional expression • evaluates WHEN conditions top-down and returns the first match — later conditions are skipped • returns <code>NULL</code> if no WHEN matches and <code>ELSE</code> is omitted.

Expression	Example	Description
Simple CASE	<pre>CASE status WHEN 'A' THEN 'Active' WHEN 'I' THEN 'Inactive' ELSE 'Unknown' END</pre>	• Equality shorthand — compares one expression to a list of values using <code>=</code> • cannot match <code>NULL</code> (because <code>NULL = NULL</code> is UNKNOWN) — use searched <code>CASE WHEN ... IS NULL</code> for that.
IIF()	<pre>IIF(salary > 50000, 'High', 'Low')</pre>	• Ternary shorthand for a two-branch CASE (SQL Server / Access) • rewritten internally as <code>CASE</code> ; nesting is capped at 10 levels, same as CASE.
CHOOSE()	<pre>CHOOSE(quarter, 'Q1', 'Q2', 'Q3', 'Q4')</pre>	• Returns the Nth item from a value list using a 1-based integer index • SQL Server only; an out-of-range or <code>NULL</code> index returns <code>NULL</code> (no error).
DECODE()	<pre>DECODE(status, 1, 'Active', 0, 'Inactive', 'Unknown')</pre>	• Oracle-proprietary equality-based CASE substitute: <code>DECODE(expr, search, result [, ...] [, default])</code> • uniquely treats two <code>NULL</code> s as equal — opposite of standard <code>CASE</code> .

TABLE 14

String Functions

String functions transform, inspect, and assemble text values directly inside SQL. Their behavior — especially around `NULL` , trailing spaces, and which dialect supports which syntax — varies meaningfully between SQL Server, PostgreSQL, MySQL, and Oracle, so the same expression can return different results depending on the engine.

Function	Example	Description
CONCAT()	<pre>CONCAT(first_name, ' ', last_name)</pre>	• Joins strings end-to-end • in SQL Server and MySQL, <code>NULL</code> arguments are converted to empty strings and ignored • PostgreSQL/Oracle/SQLite also support the <code> </code> operator, which by contrast returns <code>NULL</code> if any operand is <code>NULL</code> .
CONCAT_WS()	<pre>CONCAT_WS(' ', city, state, country)</pre>	• Concatenates with a separator — first argument is the separator • ignores <code>NULL</code> arguments and does NOT emit the separator for them (so the separator only ever appears between non-NULL values); SQL Server 2017+, PostgreSQL, MySQL.
UPPER() / LOWER()	<pre>UPPER(last_name)</pre>	• Converts a string to uppercase or lowercase according to the database's locale • case folding is locale-sensitive — the Turkish dotted/dotless I is a classic gotcha.
TRIM() / LTRIM() / RTRIM()	<pre>TRIM(LEADING ' ' FROM name)</pre>	• Removes leading, trailing, or both spaces (or specified characters) • standard SQL <code>TRIM</code> supports <code>LEADING</code> / <code>TRAILING</code> / <code>BOTH</code> • SQL Server gained <code>TRIM</code> in 2017; the <code>LEADING</code> / <code>TRAILING</code> / <code>BOTH</code> keywords require SQL Server 2022 with compatibility level 160.
SUBSTRING() / SUBSTR()	<pre>SUBSTRING(email, 1, CHARINDEX('@', email) - 1)</pre>	• Extracts a portion of a string given a 1-based start position and optional length • positions start at 1 in every major dialect; Oracle's <code>SUBSTR</code> also accepts a negative start to count from the end of the string, but SQL Server's <code>SUBSTRING</code> does not.
LEFT() / RIGHT()	<pre>LEFT(phone, 3) RIGHT(zip_code, 4)</pre>	• Extracts N characters from the left or right end of a string • a convenient shorthand for <code>SUBSTRING</code> .
LEN() / LENGTH()	<pre>WHERE LEN(description) > 500</pre>	• Returns the number of characters in a string • SQL Server's <code>LEN</code> silently excludes trailing spaces — use <code>DATALENGTH</code> if you need the byte length including them • PostgreSQL/MySQL's <code>LENGTH</code> counts everything.
REPLACE()	<pre>REPLACE(phone, '-', '')</pre>	• Substitutes all occurrences (not just the first) of a substring with another string • in SQL Server the match honors the column's collation, which is case-insensitive by default.
CHARINDEX() / INSTR() / POSITION()	<pre>CHARINDEX('@', email)</pre>	• Returns the 1-based character position of a substring, or <code>0</code> if not found (never <code>NULL</code> unless an argument is <code>NULL</code>) • <code>CHARINDEX</code> in SQL Server, <code>INSTR</code> in Oracle/MySQL, <code>POSITION(substr IN str)</code> in PostgreSQL.

Function	Example	Description
STRING_SPLIT()	<code>SELECT value FROM STRING_SPLIT('a,b,c', ',')</code>	• Splits a string on a single-character delimiter and returns a one-column table • SQL Server 2016+ • output order is not guaranteed unless you pass <code>enable_ordinal => 1</code> (SQL Server 2022+) and <code>ORDER BY ordinal</code> • PostgreSQL uses <code>string_to_table</code> / <code>regexp_split_to_table</code> .
LPAD() / RPAD()	<code>LPAD(CAST(id AS VARCHAR), 8, '0')</code> <code>RPAD(name, 20, ' ')</code>	• Pads a string to a specified length by prepending/appending fill characters; truncates from the opposite end if the input is already longer • PostgreSQL/MySQL/Oracle have these built in; SQL Server has no native <code>LPAD</code> — the usual workaround is <code>RIGHT('00000000' + CAST(id AS VARCHAR), 8)</code> .
INITCAP()	<code>INITCAP('john SMITH')</code>	• Capitalizes the first letter of each word and lowercases the rest, where "word" means a run of alphanumeric characters separated by non-alphanumerics • PostgreSQL and Oracle only — no direct equivalent in SQL Server.
FORMAT()	<code>FORMAT(amount, 'C2', 'en-US')</code>	• Formats a value as a string using .NET format strings with optional culture (SQL Server only) • supports currency, dates, and numbers with locale awareness, but relies on the CLR and is notably slower than <code>CONVERT</code> on large row counts.

TABLE 15

Date & Time Functions

Date and time handling varies more between dialects than almost any other area of SQL — function names, return types, and time-zone semantics all differ. The functions below cover the common needs: getting the current moment, doing arithmetic on dates, extracting parts, truncating to a unit, and formatting for display. Watch for two traps that bite even experienced developers: `DATEDIFF` counts boundary crossings rather than elapsed time, and `NOW()` in PostgreSQL returns the *transaction* start time, not the actual clock.

Function	Example	Description
GETDATE() / NOW() / CURRENT_TIMESTAMP	<code>SELECT GETDATE()</code>	• Returns the current date and time • <code>GETDATE()</code> in SQL Server returns server local time; use <code>GETUTCDATE()</code> or <code>SYSUTCDATETIME()</code> for UTC • <code>NOW()</code> in PostgreSQL/MySQL and <code>CURRENT_TIMESTAMP</code> (ANSI) work across dialects.
DATEADD() / DATE_ADD() / INTERVAL	<code>DATEADD(month, 3, hire_date)</code> <code>hire_date + INTERVAL '3 months'</code>	• Adds a time interval to a date; pass a negative number to subtract • <code>DATEADD</code> in SQL Server, <code>DATE_ADD</code> in MySQL, <code>+ INTERVAL</code> in PostgreSQL/Oracle.
DATEDIFF()	<code>DATEDIFF(day, start_date, end_date)</code>	• Returns the number of datepart boundaries crossed between two dates — not strictly elapsed time • <code>DATEDIFF(year, '2024-12-31', '2025-01-01')</code> returns <code>1</code> even though the dates are 1 day apart • SQL Server / MySQL syntax; PostgreSQL uses <code>EXTRACT</code> or subtraction.
DATEPART() / EXTRACT()	<code>DATEPART(year, hire_date)</code> <code>EXTRACT(YEAR FROM hire_date)</code>	• Returns a specific date component (year, month, day, hour, etc.) as an integer • <code>DATEPART</code> is T-SQL; <code>EXTRACT</code> is the ANSI-standard form supported by PostgreSQL, MySQL, Oracle, and SQL Server 2022+.
DATE_TRUNC()	<code>DATE_TRUNC('month', created_at)</code>	• Truncates a timestamp to a specified precision (year, quarter, month, day, hour) — ideal for grouping by month or quarter • PostgreSQL / Snowflake / Redshift; SQL Server 2022+ provides <code>DATETRUNC()</code> .
CAST() (to date/time)	<code>CAST('2025-01-15' AS DATE)</code>	• Converts a string to a date/datetime value (ANSI standard, works across dialects) • In SQL Server, ambiguous formats like <code>'01/02/2025'</code> depend on session <code>SET DATEFORMAT</code> / language; ISO 8601 (<code>YYYY-MM-DD</code>) is always safe.
TO_CHAR() / FORMAT()	<code>TO_CHAR(order_date, 'YYYY-MM')</code> <code>FORMAT(order_date, 'yyyy-MM')</code>	• Formats a date as a string with a template pattern — useful for display or grouping by year-month • <code>TO_CHAR</code> in PostgreSQL/Oracle/Snowflake; <code>FORMAT</code> in SQL Server 2012+ and MySQL.

Function	Example	Description
EOMONTH()	<code>EOMONTH(GETDATE())</code>	- Returns the last day of the month for a given date (as a <code>date</code>, with the time component dropped) - Optional second argument adds/subtracts months before computing month-end - SQL Server 2012+ and MySQL.
CONVERT() (date format)	<code>CONVERT(VARCHAR, GETDATE(), 103)</code>	- SQL Server's formats a date to a string using a numeric style code - Style <code>103</code> = <code>dd/mm/yyyy</code>, <code>120</code> = <code>yyyy-mm-dd hh:mi:ss</code>, <code>126</code> = ISO 8601 - SQL Server only; other dialects use <code>TO_CHAR</code> or <code>FORMAT</code>.

TABLE 16

Numeric Functions

Numeric functions handle the arithmetic, rounding, and randomness work that lives outside of plain `+` `-` `*` `/`. Dialect quirks bite hard here — what `LOG(100)` returns, whether `ROUND` rounds halves away from zero or to-even, and whether `RAND()` re-runs per row all change between vendors.

Function	Example	Description
ROUND()	<code>ROUND(123.45, -1) → 120</code>	Rounds to N decimal places - negative N rounds to tens, hundreds, etc. - SQL Server and PostgreSQL <code>numeric</code> break ties away from zero.
FLOOR() / CEILING()	<code>FLOOR(-4.7) → -5</code> <code>CEILING(4.2) → 5</code>	FLOOR returns the largest integer $\leq$ value (rounds toward $-\infty$) CEILING returns the smallest integer $\geq$ value (rounds toward $+\infty$).
ABS()	<code>ABS(balance - target)</code>	- Returns absolute value &mdash; - strips the sign - Can overflow on the minimum signed integer (e.g. <code>ABS(-2147483648)</code> in SQL Server).
POWER() / SQRT()	<code>POWER(2, 10) → 1024</code> <code>SQRT(144) → 12</code>	POWER(x, y) raises x to the y-th power SQRT returns the square root; negative input raises a domain error in most engines.
MOD() / %	<code>SELECT 17 % 5 → 2</code> <code>MOD(17, 5) → 2</code>	- Returns the remainder after division <code>%</code> in SQL Server/PostgreSQL, <code>MOD()</code> in Oracle/MySQL/PostgreSQL - In Postgres the result inherits the sign of the dividend.
LOG() / LOG10() / LN()	<code>LOG(EXP(1)) → 1</code> (SQL Server) <code>LOG(100) → 2</code> (PostgreSQL)	- SQL Server / MySQL: <code>LOG(x)</code> = natural log; <code>LOG(x, base)</code> for arbitrary base in SQL Server - PostgreSQL: <code>LOG(x)</code> = base-10; use <code>LN(x)</code> for natural log <code>LOG10</code> is always base-10.
SIGN()	<code>SIGN(balance) → -1, 0, or 1</code>	- Returns -1, 0, or +1 for negative, zero, or positive input - useful for directional logic without <code>CASE</code>.
RAND() / RANDOM()	<code>SELECT RAND()</code> (SQL Server) <code>SELECT RANDOM()</code> (PostgreSQL)	- Returns a pseudo-random float between 0 and 1 <code>RAND</code> in SQL Server/MySQL, <code>RANDOM()</code> in PostgreSQL - In SQL Server, <code>RAND()</code> evaluates once per query — use <code>NEWID()</code> for per-row randomness.
TRUNC() / TRUNCATE()	<code>TRUNC(3.987, 2) → 3.98</code> <code>TRUNC(-4.7) → -4</code>	Chops (does not round) to N decimal places - Oracle/PostgreSQL use <code>TRUNC</code>, MySQL uses <code>TRUNCATE</code> - Differs from <code>FLOOR</code> for negatives (<code>TRUNC</code> rounds toward zero, <code>FLOOR</code> toward $-\infty$).

TABLE 17

NULL Handling

SQL treats NULL as "unknown" under three-valued logic, so most operators that touch a NULL return UNKNOWN rather than TRUE or FALSE — which is why `WHERE col = NULL` never matches anything and why `NOT IN` against a subquery containing a NULL silently returns zero rows. The functions and predicates below are the safe, portable tools for testing, substituting, and aggregating NULL data across every major dialect.

Technique	Example	Description
IS NULL / IS NOT NULL	<code>WHERE phone IS NULL</code> <code>WHERE email IS NOT NULL</code>	• Tests for NULL values • the only safe way — <code>= NULL</code> and <code>!= NULL</code> always evaluate to UNKNOWN.
COALESCE()	<code>COALESCE(mobile, home_phone, 'N/A')</code>	• Returns the first non-NULL argument (NULL only if all are NULL) • ANSI standard • accepts any number of arguments • short-circuits left-to-right.
NULLIF()	<code>NULLIF(denominator, 0)</code>	• Returns NULL if both arguments are equal, otherwise the first argument • classic use prevents division-by-zero: <code>value / NULLIF(denom, 0)</code>.
ISNULL() / IFNULL() / NVL()	<code>ISNULL(discount, 0)</code>	• Two-argument NULL substitution (vendor-specific) • SQL Server <code>ISNULL</code>, MySQL <code>IFNULL</code>, Oracle <code>NVL</code> • less portable than COALESCE.
NULL in aggregates	<code>AVG(score)</code> (NULL ignored) <code>COUNT(*)</code> (NULL included)	• Most aggregates skip NULLs in both numerator and denominator • <code>COUNT(*)</code> counts all rows; <code>COUNT(col)</code> skips NULLs.
NULL in JOINS	<code>a LEFT JOIN b ON a.id = b.id</code> <code>WHERE b.id IS NULL -- anti-join</code>	• NULL keys never match in join conditions • filtering an outer-join column in <code>WHERE</code> quietly converts LEFT JOIN to INNER JOIN • <code>NOT IN (subquery)</code> returns zero rows if the subquery yields any NULL — use <code>NOT EXISTS</code> instead.

TABLE 18

Pattern Matching

SQL provides several pattern-matching predicates for filtering rows by text shape — from the simple wildcard-based `LIKE` to dialect-specific regex operators and full-text search. Which one you reach for depends on how complex the pattern is and which engine you're on.

Technique	Example	Description
LIKE	<code>WHERE name LIKE 'A%'</code> <code>WHERE code LIKE '_-[0-9][0-9]'</code>	• <code>%</code> matches any sequence of characters; <code>_</code> matches exactly one character • SQL Server also supports character classes <code>[a-z]</code>.
ILIKE	<code>WHERE name ILIKE '%smith%'</code>	• Case-insensitive LIKE • PostgreSQL only • equivalent to <code>LIKE</code> with <code>LOWER()</code>.
NOT LIKE	<code>WHERE email NOT LIKE '%@example.com'</code>	Inverts pattern match — returns rows where the value does not match the pattern.
REGEXP_LIKE / SIMILAR TO / ~ (tilde)	<code>REGEXP_LIKE(phone, '^d{3}-d{4}\$')</code> (Oracle/MySQL) <code>phone ~ '^d{3}-d{4}\$'</code> (PostgreSQL)	• Full regular expression matching • <code>REGEXP_LIKE</code> in Oracle/MySQL, <code>~</code> in PostgreSQL, <code>SIMILAR TO</code> is SQL standard but limited; <code>REGEXP_MATCH</code> in PostgreSQL extracts matches.
CONTAINS	<code>WHERE CONTAINS(description, 'fast AND reliable')</code>	• Full-text search predicate (SQL Server) • requires a full-text index; supports boolean operators and proximity searches.
ESCAPE clause	<code>WHERE path LIKE '100%' ESCAPE '\'</code>	Treats special LIKE metacharacters (<code>%</code> , <code>_</code>) as literals when prefixed with the escape character.

TABLE 19

Data Type Conversion

Explicit conversion functions turn one data type into another and decide what happens when the input doesn't fit. The big split is **error vs NULL on failure**: `CAST` / `CONVERT` / `PARSE` raise errors, while `TRY_CAST` / `TRY_CONVERT` / `TRY_PARSE` return NULL — much safer for dirty user input. Choosing the right one (and pairing column types in `WHERE` filters) also has real performance impact, because implicit conversions can stop SQL Server from using an index.

Function	Example	Description
CAST()	<code>CAST('3.14' AS DECIMAL(10,2))</code>	• ANSI-standard explicit type conversion — works in T-SQL, PostgreSQL, MySQL, Oracle, Snowflake, DuckDB • raises an error if the conversion fails.
CONVERT()	<code>CONVERT(VARCHAR, GETDATE(), 120)</code>	• SQL Server type conversion with an optional <code>style</code> argument for date/number formatting (e.g., 120 = ODBC canonical <code>yyyy-mm-dd hh:mi:ss</code>) • MySQL <code>CONVERT</code> has no style code.
TRY_CAST()	<code>TRY_CAST(user_input AS INT)</code>	• Safe CAST (SQL Server 2012+) — returns NULL instead of an error when the conversion fails • ideal for validating user input or dirty data.
TRY_CONVERT()	<code>TRY_CONVERT(DATE, date_string, 101)</code>	• Safe CONVERT (SQL Server 2012+) — returns NULL on conversion failure • supports the same style codes as <code>CONVERT</code>.
TRY_PARSE()	<code>TRY_PARSE('€1.234,56' AS MONEY USING 'de-DE')</code>	• Culture-aware string parsing to date/time or number types — returns NULL on failure • relies on the .NET CLR, so slower than <code>TRY_CAST</code>.
TO_NUMBER() / TO_DATE() / TO_TIMESTAMP()	<code>TO_DATE('05 Dec 2000', 'DD Mon YYYY')</code> <code>TO_NUMBER('12,454.8', '99G999D9')</code>	• PostgreSQL and Oracle format-model conversions • the explicit format string controls how the input is parsed.
PARSE()	<code>PARSE('15 Janvier 2025' AS DATE USING 'fr-FR')</code>	• SQL Server culture-aware string-to-date/number parse; raises an error on failure (use <code>TRY_PARSE</code> for the safe variant) • depends on the .NET CLR.
STR()	<code>STR(123.45, 6, 1)</code> returns <code>'123.5'</code>	• SQL Server float-to-character conversion with fixed length and decimal places • right-justifies with leading spaces and returns asterisks (<code>**</code>) when the length is too small.

TABLE 20

Constraints & Keys

Constraints enforce data integrity at the column or table level, rejecting writes that would violate the rules you declare in `CREATE TABLE` or `ALTER TABLE`. Watch the NULL handling closely — UNIQUE, CHECK, and FOREIGN KEY each treat NULL differently, and the defaults differ across SQL Server, PostgreSQL, MySQL, and Oracle.

Constraint	Example	Description
PRIMARY KEY	<code>id INT PRIMARY KEY</code>	• Uniquely identifies each row — enforces NOT NULL + UNIQUE together • one per table; may span multiple columns (composite PK) • creates a unique clustered index by default in SQL Server (override with <code>PRIMARY KEY NONCLUSTERED</code>); PostgreSQL has no clustered-index concept.
FOREIGN KEY	<code>FOREIGN KEY (dept_id) REFERENCES departments(id)</code>	Enforces referential integrity — the value must match a row in the referenced table or be NULL (FK columns are nullable unless you also add <code>NOT NULL</code>).
NOT NULL	<code>name VARCHAR(100) NOT NULL</code>	• Rejects NULL values on INSERT and UPDATE • independent from <code>DEFAULT</code> — supplying an explicit NULL still raises an error even if a default exists.

Constraint	Example	Description
UNIQUE	<code>email VARCHAR(255) UNIQUE</code>	- Ensures all non-NULL values are distinct and creates a backing unique index - PostgreSQL / MySQL / Oracle allow multiple NULLs (NULL ≠ NULL); SQL Server's basic UNIQUE allows only one NULL — use a filtered unique index (<code>WHERE col IS NOT NULL</code>) for multi-NULL uniqueness.
CHECK	<code>salary DECIMAL CHECK (salary >= 0)</code>	Validates a Boolean expression on INSERT and UPDATE - NULL is treated as UNKNOWN, not FALSE — a CHECK constraint passes when the expression is TRUE or NULL, so NULLs slip through unless you also add <code>NOT NULL</code> .
DEFAULT	<code>created_at DATETIME DEFAULT GETDATE()</code>	Supplies a value when the column is omitted from INSERT or you write the <code>DEFAULT</code> keyword - an explicit NULL is NOT replaced by the default — it goes in as NULL (or errors if the column is NOT NULL).
ON DELETE CASCADE	<code>FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE</code>	Automatically deletes child rows when the referenced parent row is deleted - alternatives: <code>SET NULL</code> nullifies the child FK column, <code>SET DEFAULT</code> uses the child's default, <code>NO ACTION / RESTRICT</code> block the parent delete.

TABLE 21

Indexes

Indexes are the single biggest performance lever in SQL: they turn full table scans into fast lookups by keeping a sorted, pointer-based structure (usually a B-tree) over chosen columns. They are pure redundancy though — every INSERT, UPDATE, or DELETE that touches an indexed column also rewrites the index — so over-indexing is just as damaging as missing indexes, and column order in multi-column indexes is rarely a free choice.

Concept	Example	Description
CREATE INDEX	<code>CREATE INDEX idx_last_name ON customers (last_name)</code>	Creates a non-unique index (B-tree by default) to speed up queries that filter, sort, or join on the column(s). Indexes are redundant copies — they cost storage and slow writes.
CREATE UNIQUE INDEX	<code>CREATE UNIQUE INDEX idx_email ON users (email)</code>	- Enforces uniqueness on the indexed column(s) while accelerating lookups - functionally equivalent to a UNIQUE constraint (which is implemented as a unique index behind the scenes).
Clustered Index	<code>CREATE CLUSTERED INDEX idx_order_id ON orders (order_id)</code>	Physically orders the table rows by the index key — the table data IS the clustered index's leaf level - only one per table; in SQL Server, PRIMARY KEY creates a clustered index by default. PostgreSQL has no clustered-index concept (only a one-time <code>CLUSTER</code> reorder command).
Non-clustered Index	<code>CREATE NONCLUSTERED INDEX idx_dept ON employees (department_id)</code>	- Maintains a separate structure with key values plus row locators back to the heap or clustered index - multiple allowed per table; fast for selective lookups but each non-covered query needs a key lookup.
Composite Index	<code>CREATE INDEX idx_name_dept ON employees (last_name, department_id)</code>	- Index on multiple columns in a defined order — usable only when the query filters on a leftmost prefix of those columns - an index on (A, B) helps <code>WHERE A=...</code> and <code>WHERE A=... AND B=...</code> , but not <code>WHERE B=...</code> alone.
Covering Index (INCLUDE)	<code>CREATE INDEX idx_cust ON orders (customer_id) INCLUDE (amount, order_date)</code>	- Adds non-key columns to the index leaf via <code>INCLUDE</code> (SQL Server, PostgreSQL 11+) - query is satisfied entirely from the index with no key lookup back to the table — a big read-path win for hot queries.
Filtered Index (Partial Index)	<code>CREATE INDEX idx_active ON users (email) WHERE active = 1</code>	- Indexes only the rows matching a WHERE predicate ("filtered index" in SQL Server, "partial index" in PostgreSQL) - smaller, cheaper to maintain, and useful when queries usually target a known subset.

Concept	Example	Description
DROP INDEX	<code>DROP INDEX idx_last_name ON customers</code>	• Removes an index and frees its storage • syntax varies: SQL Server uses <code>DROP INDEX name ON table</code>; PostgreSQL and MySQL use <code>DROP INDEX name</code> (MySQL also supports <code>ALTER TABLE ... DROP INDEX</code>). Cannot drop indexes backing a PRIMARY KEY or UNIQUE constraint — drop the constraint instead.

TABLE 22

Views

Views package a SELECT statement under a name so it can be queried like a table. Standard views store no data — the underlying query re-runs on every reference, which is cheap on storage but offers no speed-up over the raw query. Materialized views flip that trade-off by persisting the result on disk for fast reads at the cost of staleness until the next refresh.

Technique	Example	Description
CREATE VIEW	<code>CREATE VIEW active_customers AS SELECT * FROM customers WHERE active = 1</code>	• Creates a named, saved query that can be queried like a table • not physically materialized — the defining query is re-run on every reference, so storage is zero but there is no performance gain over the underlying SELECT.
CREATE OR REPLACE VIEW	<code>CREATE OR REPLACE VIEW monthly_sales AS SELECT EXTRACT(MONTH FROM order_date) AS month, SUM(amount) FROM orders GROUP BY 1</code>	• Atomically replaces an existing view's defining query without dropping it • PostgreSQL, MySQL, Oracle • the new query must produce the same column names, order, and types — extra columns may only be appended at the end • SQL Server uses <code>ALTER VIEW</code> or <code>CREATE OR ALTER VIEW</code> (2016 SP1+) instead.
WITH CHECK OPTION	<code>CREATE VIEW eng_staff AS SELECT * FROM employees WHERE dept = 'Engineering' WITH CHECK OPTION</code>	• Rejects INSERT, UPDATE, or MERGE through the view if the resulting row would no longer satisfy the view's WHERE clause • <code>LOCAL</code> checks only the current view; <code>CASCADE</code> (the default if neither is specified) also checks every underlying view.
Materialized View	<code>CREATE MATERIALIZED VIEW report_summary AS SELECT dept, SUM(salary) FROM employees GROUP BY dept; REFRESH MATERIALIZED VIEW CONCURRENTLY report_summary;</code>	• Stores query results physically — SELECTs are fast but data is stale until the next <code>REFRESH MATERIALIZED VIEW</code> • a plain refresh takes an exclusive lock; <code>CONCURRENTLY</code> (PG 9.4+) avoids blocking readers but requires a UNIQUE index on the view • PostgreSQL and Oracle; SQL Server's equivalent is an indexed view (auto-maintained on every base-table change, no manual refresh).
DROP VIEW	<code>DROP VIEW IF EXISTS active_customers</code>	• Removes only the view definition — underlying tables and their rows are unaffected • <code>RESTRICT</code> (the default) blocks the drop if other objects depend on the view; <code>CASCADE</code> drops them too.

TABLE 23

Transactions

Transactions group one or more SQL statements into an atomic unit so the database can guarantee ACID properties (Atomicity, Consistency, Isolation, Durability). The statements below mark transaction boundaries, set the visibility rules between concurrent sessions, and (in the case of `WITH (NOLOCK)`) trade correctness for raw read speed in SQL Server.

Statement	Example	Description
BEGIN TRANSACTION	<code>BEGIN TRANSACTION</code>	• Starts an explicit transaction block — subsequent changes are held until COMMIT or ROLLBACK • <code>BEGIN</code> / <code>START TRANSACTION</code> in PostgreSQL and MySQL; most engines default to autocommit (each statement is its own transaction) unless wrapped.
COMMIT	<code>COMMIT</code>	• Permanently saves all changes made since the transaction started and frees the transaction's locks and resources • cannot be undone — once committed, ROLLBACK no longer applies.

Statement	Example	Description
ROLLBACK	ROLLBACK	- Discards all changes made since BEGIN (or since a named SAVEPOINT, with ROLLBACK TO) - used for error recovery and to abort transactions cleanly.
SAVEPOINT	SAVEPOINT before_update ROLLBACK TO before_update	- Creates a named checkpoint inside a transaction - ROLLBACK TO savepoint undoes work back to that point without ending the transaction — the outer transaction stays open and can still COMMIT.
SET TRANSACTION ISOLATION LEVEL	SET TRANSACTION ISOLATION LEVEL READ COMMITTED	- Controls which concurrency anomalies are allowed — dirty reads, non-repeatable reads, phantom reads - Levels (ascending strictness): READ UNCOMMITTED → READ COMMITTED → REPEATABLE READ → SERIALIZABLE; SQL Server adds SNAPSHOT. Defaults vary: READ COMMITTED in SQL Server / PostgreSQL / Oracle, REPEATABLE READ in MySQL InnoDB.
WITH (NOLOCK)	SELECT * FROM orders WITH (NOLOCK)	- SQL Server table hint equivalent to READ UNCOMMITTED for that table — reads without acquiring shared locks - can return uncommitted (dirty) data, duplicate rows, or miss previously-committed rows; a notorious anti-pattern when used as a "go-fast" switch.

TABLE 24

Error Handling

SQL Server's structured error handling centers on `TRY...CATCH` : any runtime error of severity above 10 inside the TRY block jumps execution to the matching CATCH block, where a family of `ERROR_*` functions exposes the message, number, severity, line, and procedure of the original error. Inside CATCH you also check `XACT_STATE()` to decide whether to commit or roll back, and re-surface errors with `THROW` (preferred since SQL Server 2012) or the older `RAISERROR` .

Construct	Example	Description
TRY...CATCH	BEGIN TRY INSERT INTO orders VALUES (1, 99.99) END TRY BEGIN CATCH SELECT ERROR_MESSAGE(), ERROR_NUMBER() END CATCH	Structured error handling (SQL Server / Azure SQL) - any runtime error of severity > 10 in TRY transfers control to CATCH; compile and name-resolution errors at the same scope are NOT caught.
ERROR_MESSAGE()	SELECT ERROR_MESSAGE()	- Returns the error message text of the error that triggered the CATCH block - returns NULL outside CATCH
ERROR_NUMBER()	SELECT ERROR_NUMBER()	- Returns the error number — correlates to <code>sys.messages</code> for system errors - user-defined errors use 50000+ - unlike <code>@@ERROR</code> , the value persists across the whole CATCH scope.
XACT_STATE()	IF XACT_STATE() = -1 ROLLBACK IF XACT_STATE() = 1 COMMIT	- Returns 1 (active committable), -1 (active uncommittable — must rollback), or 0 (no active transaction) - essential check in CATCH blocks before deciding to COMMIT or ROLLBACK.
THROW	THROW 50001, 'Invalid customer ID', 1 -- or to re-throw: THROW	- Raises a user-defined error (number ≥ 50000, severity always 16) or re-throws the original error with its original details (bare <code>THROW</code> in CATCH) - SQL Server 2012+; preferred over <code>RAISERROR</code> for new code; honors <code>SET XACT_ABORT</code> .
RAISERROR()	RAISERROR('Custom error: %s', 16, 1, @detail)	Older mechanism for raising errors; supports custom severity 1-25 and printf-style message formatting (<code>%s</code> , <code>%d</code>) - does not honor <code>SET XACT_ABORT</code> ; still useful when you need a custom severity.

TABLE 25

JSON Functions

SQL Server stores JSON as plain `nvarchar` (or the native `json` type in 2025+) and exposes a family of functions to extract, validate, build, and shred it. Each function pairs with a specific shape — scalars use `JSON_VALUE`, objects and arrays use `JSON_QUERY`, and rowset shredding uses `OPENJSON` — and most accept a `lax` (silent NULL) or `strict` (raise error) path mode.

Function	Example	Description
JSON_VALUE()	<code>JSON_VALUE(payload, '\$.customer.name')</code>	- Extracts a scalar value (string, number, boolean) from a JSON path - returns NULL in lax mode (default) if the path is missing or points to an object/array.
JSON_QUERY()	<code>JSON_QUERY(payload, '\$.address')</code>	- Extracts a JSON object or array fragment - returns NULL for scalar values in lax mode (the opposite of <code>JSON_VALUE</code>).
JSON_OBJECT()	<code>JSON_OBJECT('id':id, 'name':name)</code>	Constructs a JSON object from key:value pairs (SQL Server 2022+) - default NULL ON NULL keeps NULL keys; ABSENT ON NULL omits them.
JSON_ARRAY()	<code>JSON_ARRAY(1, 'two', NULL)</code>	Constructs a JSON array from values (SQL Server 2022+) - default ABSENT ON NULL silently drops NULL elements (opposite of <code>JSON_OBJECT</code> default).
ISJSON()	<code>WHERE ISJSON(payload) = 1</code>	- Returns 1 if the string is valid JSON, 0 if not, NULL if input is NULL 2022+ accepts a type constraint: VALUE, ARRAY, OBJECT, SCALAR.
JSON_MODIFY()	<code>JSON_MODIFY(payload, '\$.status', 'active')</code>	- Returns a new JSON string with one property updated - setting NULL in lax mode deletes the key - use the <code>append</code> keyword to push onto an array.
OPENJSON()	<code>SELECT * FROM OPENJSON(@json) WITH (id INT, name VARCHAR(50))</code>	Parses JSON into a rowset (SQL Server 2016+) - without WITH returns generic key/value/type columns; WITH defines typed columns and supports <code>AS JSON</code> for nested fragments.

TABLE 26

Query Execution Order

The order in which you write SQL clauses is not the order the database evaluates them. Understanding this logical processing order is the single biggest unlock for "why doesn't this work?" errors — it explains why `WHERE` cannot see `SELECT` aliases, why `GROUP BY` rejects non-aggregated columns, and why window functions cannot be filtered in `WHERE`.

Step	Example	Description
1. FROM / JOIN	<code>FROM orders o JOIN customers c ON o.customer_id = c.id</code>	First step — identifies source tables and performs joins - table aliases defined here are available to every later clause.
2. WHERE	<code>WHERE o.status = 'shipped'</code>	Filters individual rows before any grouping - cannot reference <code>SELECT</code> aliases, aggregate functions, or window functions.
3. GROUP BY	<code>GROUP BY c.country</code>	Collapses rows into groups for aggregate computation — every non-aggregated column in <code>SELECT</code> must appear here.
4. HAVING	<code>HAVING COUNT(o.id) > 5</code>	Filters groups after aggregation - can reference aggregate functions; cannot reference <code>SELECT</code> aliases in standard SQL.
5. SELECT (+ window functions)	<code>SELECT c.country, COUNT(o.id), SUM(o.amount)</code>	Evaluates expressions, aliases, and window functions — aliases defined here are why <code>WHERE</code>/<code>GROUP BY</code>/<code>HAVING</code> can't see them.
6. QUALIFY	<code>QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) = 1</code>	Filters on window function results — what <code>HAVING</code> is to <code>GROUP BY</code>, <code>QUALIFY</code> is to window functions - Snowflake, BigQuery, DuckDB, Databricks; not ANSI standard.

Step	Example	Description
7. DISTINCT	<code>SELECT DISTINCT country</code>	Removes duplicate rows from the result set after SELECT expressions are evaluated.
8. ORDER BY	<code>ORDER BY SUM(o.amount) DESC</code>	• Sorts the final result set — runs after SELECT, so it CAN reference SELECT aliases • without it, row order is not guaranteed.
9. LIMIT / OFFSET / TOP / FETCH	<code>LIMIT 10 OFFSET 20</code>	• Restricts and paginates the already-sorted result • without ORDER BY the rows returned are unpredictable.

TABLE 27

Advanced Ordering

Beyond a single `ORDER BY column ASC`, real queries often need per-column sort directions, explicit NULL placement, custom priority orders, locale-aware comparison, or row counts that include ties. The exact syntax and defaults vary across SQL dialects, so portable code typically spells out direction, NULL handling, and tie-handling explicitly rather than relying on the database default.

Technique	Example	Description
ORDER BY ASC / DESC	<code>ORDER BY salary DESC, name ASC</code>	• Multi-column sort — secondary columns resolve ties from the primary column • ASC is the default; direction is per-column, so <code>ORDER BY a, b DESC</code> sorts <code>a</code> ASC then <code>b</code> DESC.
NULLS FIRST / NULLS LAST	<code>ORDER BY score DESC NULLS LAST</code>	• Explicit NULL positioning in the sort (ANSI standard) • defaults differ — PostgreSQL/Oracle treat NULL as larger (NULLS LAST for ASC); SQL Server, MySQL, SQLite treat NULL as smaller • SQL Server does not support the keyword — emulate with <code>CASE WHEN col IS NULL THEN 1 ELSE 0 END</code>.
ORDER BY column position	<code>ORDER BY 1, 2</code>	• Reference by SELECT-list position — 1 = first selected column • convenient for ad-hoc queries but fragile in production: reordering the SELECT list silently changes the sort.
CASE in ORDER BY	<code>ORDER BY CASE status WHEN 'urgent' THEN 1 WHEN 'normal' THEN 2 ELSE 3 END</code>	• Custom sort priority using a CASE expression as the sort key • enables business-rule ordering that doesn't follow alphabetical or numeric order.
COLLATE	<code>ORDER BY name COLLATE Latin1_General_CI_AI</code>	• Overrides the column's collation for that one sort — CI = case-insensitive, AI = accent-insensitive • useful when the column's default collation doesn't match the sort you need (e.g., locale-aware alphabetisation).
TOP WITH TIES	<code>SELECT TOP 3 WITH TIES name, score FROM leaderboard ORDER BY score DESC</code>	• Includes rows tied with the last qualifying row's ORDER BY value, so the result may exceed N • requires <code>ORDER BY</code>; SQL Server syntax — PostgreSQL uses the SQL-standard <code>FETCH FIRST n ROWS WITH TIES</code>.

References

Articles, guides, & named-author resources

1. FROM / JOIN - <https://www.postgresql.org/docs/current/sql-select.html#SQL-FROM>
2. 3. GROUP BY - <https://www.postgresql.org/docs/current/sql-select.html#SQL-GROUPBY>
3. 5. SELECT (+ window functions) - <https://www.postgresql.org/docs/current/sql-select.html#SQL-SELECT-LIST>
4. 6. QUALIFY - <https://docs.snowflake.com/en/sql-reference/constructs/qualify>
5. 7. DISTINCT - <https://www.postgresql.org/docs/current/sql-select.html#SQL-DISTINCT>
6. 8. ORDER BY - <https://learn.microsoft.com/en-us/sql/t-sql/queries/select-order-by-clause-transact-sql>
7. ABS() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/abs-transact-sql>
8. ALTER TABLE ADD COLUMN - <https://www.postgresql.org/docs/current/sql-altertable.html>
9. ALTER TABLE MODIFY / ALTER COLUMN - <https://learn.microsoft.com/en-us/sql/t-sql/statements/alter-table-transact-sql>
10. AVG() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/avg-transact-sql?view=sql-server-ver17>
11. AVG() OVER() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/avg-transact-sql>
12. BEGIN TRANSACTION - <https://learn.microsoft.com/en-us/sql/t-sql/language-elements/begin-transaction-transact-sql?view=sql-server-ver17>

13. CASE in ORDER BY - <https://learn.microsoft.com/en-us/sql/t-sql/queries/select-order-by-clause-transact-sql?view=sql-server-ver17#specify-a-conditional-order>
15. CAST() (to date/time) - <https://learn.microsoft.com/en-us/sql/t-sql/functions/cast-and-convert-transact-sql?view=sql-server-ver17>
17. CHOOSE() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/logical-functions-choose-transact-sql?view=sql-server-ver17>
19. COLLATE - <https://learn.microsoft.com/en-us/sql/t-sql/queries/select-order-by-clause-transact-sql?view=sql-server-ver17#specify-a-collation>
21. Composite Index - <https://use-the-index-luke.com/sql/where-clause/the-equals-operator/concatenated-keys>
23. CONCAT_WS() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/concat-ws-transact-sql?view=sql-server-ver17>
25. Correlated Subquery - <https://duckdb.org/2023/05/26/correlated-subqueries-in-sql>
27. COUNT() OVER() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/count-transact-sql>
29. CREATE DATABASE - <https://www.postgresql.org/docs/current/sql-createdatabase.html>
31. CREATE TABLE - <https://www.postgresql.org/docs/current/sql-createtable.html>
33. CREATE VIEW - <https://www.postgresql.org/docs/current/sql-createview.html>
35. CTE (Common Table Expression) - <https://www.postgresql.org/docs/current/queries-with.html>
37. DATE_TRUNC() - <https://www.postgresql.org/docs/current/functions-datetime.html>
39. DATEDIFF() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/datediff-transact-sql?view=sql-server-ver17>
41. DECODE() - <https://docs.oracle.com/en/database/oracle/oracle-database/21/sqlrf/DECODE.html>
43. DELETE FROM - <https://www.postgresql.org/docs/current/sql-delete.html>
45. DISTINCT - <https://www.postgresql.org/docs/current/queries-select-lists.html#QUERIES-DISTINCT>
47. DROP TABLE - <https://www.postgresql.org/docs/current/sql-droptable.html>
49. Dynamic PIVOT (Conditional Aggregation) - <https://database.guide/how-to-pivot-data-in-sql-without-the-pivot-operator/>
51. ERROR_MESSAGE() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/error-message-transact-sql?view=sql-server-ver17>
53. Filtered Index (Partial Index) - <https://www.postgresql.org/docs/current/indexes-partial.html>
55. FLOOR() / CEILING() - <https://www.postgresql.org/docs/current/functions-math.html>
57. FORMAT() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/format-transact-sql?view=sql-server-ver17>
59. GETDATE() / NOW() / CURRENT_TIMESTAMP - <https://learn.microsoft.com/en-us/sql/t-sql/functions/current-timestamp-transact-sql?view=sql-server-ver17>
61. GROUP_CONCAT / STRING_AGG - <https://learn.microsoft.com/en-us/sql/t-sql/functions/string-aggr-transact-sql?view=sql-server-ver17>
63. GROUPING_ID() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/grouping-id-transact-sql?view=sql-server-ver17>
65. IIF() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/logical-function-s-iif-transact-sql?view=sql-server-ver17>
67. INSERT INTO ... VALUES - <https://www.postgresql.org/docs/current/sql-insert.html>
69. IS NULL / IS NOT NULL - <https://www.postgresql.org/docs/current/functions-comparison.html>
71. ISNULL() / IFNULL() / NVL() - <https://learn.microsoft.com/en-us/sql/t-sql/language-elements/coalesce-transact-sql?view=sql-server-ver17>
73. JSON_MODIFY() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/json-modify-transact-sql?view=sql-server-ver17>
14. CASE WHEN ... THEN ... END - <https://www.postgresql.org/docs/current/functions-conditional.html>
16. CHARINDEX() / INSTR() / POSITION() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/charindex-transact-sql?view=sql-server-ver17>
18. Clustered Index - <https://learn.microsoft.com/en-us/sql/relational-databases/indexes/clustered-and-nonclustered-indexes-described?view=sql-server-ver17>
20. COMMIT - <https://www.postgresql.org/docs/current/sql-commit.html>
22. CONCAT() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/concat-transact-sql?view=sql-server-ver17>
24. CONTAINS - <https://learn.microsoft.com/en-us/sql/t-sql/queries/contains-transact-sql?view=sql-server-ver17>
26. COUNT() - <https://www.postgresql.org/docs/current/functions-aggregate.html>
28. Covering Index (INCLUDE) - <https://learn.microsoft.com/en-us/sql/relational-databases/indexes/create-indexes-with-included-columns?view=sql-server-ver17>
30. CREATE INDEX - <https://www.postgresql.org/docs/current/sql-createindex.html>
32. CREATE UNIQUE INDEX - <https://learn.microsoft.com/en-us/sql/relational-databases/indexes/create-unique-indexes?view=sql-server-ver17>
34. CROSS APPLY / OUTER APPLY - <https://learn.microsoft.com/en-us/sql/t-sql/queries/from-transact-sql?view=sql-server-ver17>
36. CUME_DIST() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/cume-dist-transact-sql?view=sql-server-ver17>
38. DATEADD() / DATE_ADD() / INTERVAL - <https://learn.microsoft.com/en-us/sql/t-sql/functions/dateadd-transact-sql?view=sql-server-ver17>
40. DATEPART() / EXTRACT() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/datepart-transact-sql?view=sql-server-ver17>
42. DEFAULT - <https://learn.microsoft.com/en-us/sql/t-sql/statements/create-table-transact-sql?view=sql-server-ver17>
44. DENSE_RANK() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/dense-rank-transact-sql?view=sql-server-ver17>
46. DROP INDEX - <https://learn.microsoft.com/en-us/sql/t-sql/statements/drop-index-transact-sql?view=sql-server-ver17>
48. DROP VIEW - <https://www.postgresql.org/docs/current/sql-dropview.html>
50. EOMONTH() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/eomonth-transact-sql?view=sql-server-ver17>
52. ERROR_NUMBER() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/error-number-transact-sql?view=sql-server-ver17>
54. FIRST_VALUE() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/first-value-transact-sql>
56. FOREIGN KEY - <https://learn.microsoft.com/en-us/sql/relational-databases/tables/primary-and-foreign-key-constraints?view=sql-server-ver17>
58. FROM - <https://www.postgresql.org/docs/current/queries-table-expressions.html>
60. GROUP BY - <https://www.postgresql.org/docs/current/queries-table-expressions.html#QUERIES-GROUP>
62. GROUPING() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/grouping-transact-sql?view=sql-server-ver17>
64. HAVING - <https://www.postgresql.org/docs/current/sql-select.html#SQL-HAVING>
66. ILIKE - <https://www.postgresql.org/docs/current/functions-matching.html>
68. INTERSECT - <https://learn.microsoft.com/en-us/sql/t-sql/language-elements/set-operators-except-and-intersect-transact-sql?view=sql-server-ver17>
70. ISJSON() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/isjson-transact-sql?view=sql-server-ver17>
72. JSON_ARRAY() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/json-array-transact-sql?view=sql-server-ver17>
74. JSON_OBJECT() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/json-object-transact-sql?view=sql-server-ver17>

75. JSON_QUERY() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/json-query-transact-sql?view=sql-server-ver17>
77. LAG() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/lag-transact-sql>
79. LEAD() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/lead-transact-sql>
81. LEN() / LENGTH() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/len-transact-sql?view=sql-server-ver17>
83. LIMIT / TOP / FETCH FIRST - <https://www.postgresql.org/docs/current/queries-limit.html>
85. Materialized View - <https://www.postgresql.org/docs/current/rules-materializedviews.html>
87. MERGE (UPsert) - <https://learn.microsoft.com/en-us/sql/t-sql/statements/merge-transact-sql?view=sql-server-ver17>
89. N PRECEDING / N FOLLOWING - <https://learn.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql>
91. NTH_VALUE() - <https://www.postgresql.org/docs/current/functions-window.html>
93. ON DELETE CASCADE - <https://learn.microsoft.com/en-us/sql/relational-databases/tables/primary-and-foreign-key-constraints?view=sql-server-ver17#cascading-referential-integrity>
95. ORDER BY - <https://www.postgresql.org/docs/current/queries-order.html>
97. PARSE() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/parse-transact-sql?view=sql-server-ver17>
99. PIVOT - <https://learn.microsoft.com/en-us/sql/t-sql/queries/from-using-pivot-and-unpivot?view=sql-server-ver17>
101. PRIMARY KEY - <https://learn.microsoft.com/en-us/sql/relational-databases/tables/create-primary-keys?view=sql-server-ver17>
103. RAND() / RANDOM() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/rand-transact-sql>
105. Recursive CTE - <https://www.postgresql.org/docs/current/queries-with.html#QUERIES-WITH-RECURSIVE>
107. REPLACE() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/replace-transact-sql?view=sql-server-ver17>
109. ROUND() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/round-transact-sql>
111. ROWS BETWEEN ... AND ... - <https://www.postgresql.org/docs/current/sql-expressions.html#SYNTAX-WINDOW-FUNCTIONS>
113. Scalar Subquery - <https://www.postgresql.org/docs/current/sql-expression-s.html#SQL-SYNTAX-SCALAR-SUBQUERIES>
115. SET TRANSACTION ISOLATION LEVEL - <https://learn.microsoft.com/en-us/sql/t-sql/statements/set-transaction-isolation-level-transact-sql?view=sql-server-ver17>
117. Simple CASE - <https://learn.microsoft.com/en-us/sql/t-sql/language-elements/case-transact-sql?view=sql-server-ver17>
119. STR() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/str-transact-sql?view=sql-server-ver17>
121. Subquery (WHERE) - <https://www.postgresql.org/docs/current/functions-subquery.html>
123. SUM() OVER() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/sum-transact-sql>
125. TO_CHAR() / FORMAT() - <https://www.postgresql.org/docs/current/functions-formatting.html>
127. TRIM() / LTRIM() / RTRIM() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/trim-transact-sql?view=sql-server-ver17>
129. TRY...CATCH - <https://learn.microsoft.com/en-us/sql/t-sql/language-elements/try-catch-transact-sql?view=sql-server-ver17>
131. TRY_CONVERT() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/try-convert-transact-sql?view=sql-server-ver17>
133. UNION - <https://www.postgresql.org/docs/current/queries-union.html>
135. UPDATE ... SET - <https://www.postgresql.org/docs/current/sql-update.html>
137. UPPER() / LOWER() - <https://www.postgresql.org/docs/current/functions-string.html>
76. JSON_VALUE() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/json-value-transact-sql?view=sql-server-ver17>
78. LAST_VALUE() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/last-value-transact-sql>
80. LEFT() / RIGHT() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/left-transact-sql?view=sql-server-ver17>
82. LIKE - <https://learn.microsoft.com/en-us/sql/t-sql/language-elements/like-transact-sql?view=sql-server-ver17>
84. LOG() / LOG10() / LN() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/log-transact-sql>
86. MAX() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/max-transact-sql?view=sql-server-ver17>
88. MIN() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/min-transact-sql?view=sql-server-ver17>
90. NOT NULL - <https://www.postgresql.org/docs/current/ddl-constraints.html#DDL-CONSTRAINTS-NOT-NULL>
92. NTILE() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/ntile-transact-sql?view=sql-server-ver17>
94. OPENJSON() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/openjson-transact-sql?view=sql-server-ver17>
96. ORDER BY column position - <https://learn.microsoft.com/en-us/sql/t-sql/queries/select-order-by-clause-transact-sql?view=sql-server-ver17>
98. PERCENT_RANK() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/percent-rank-transact-sql?view=sql-server-ver17>
100. POWER() / SQRT() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/power-transact-sql>
102. RAISERROR() - <https://learn.microsoft.com/en-us/sql/t-sql/language-elements/raiserror-transact-sql?view=sql-server-ver17>
104. RANK() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/rank-transact-sql?view=sql-server-ver17>
106. REGEXP_LIKE / SIMILAR TO / ~ (tilde) - https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/REGEXP_LIKE-Condition.html
108. ROLLBACK - <https://www.postgresql.org/docs/current/sql-rollback.html>
110. ROW_NUMBER() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/row-number-transact-sql?view=sql-server-ver17>
112. SAVEPOINT - <https://www.postgresql.org/docs/current/sql-savepoint.html>
114. SELECT - <https://learn.microsoft.com/en-us/sql/t-sql/queries/select-transact-sql>
116. SIGN() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/sign-transact-sql>
118. STDDEV / STDEV - <https://learn.microsoft.com/en-us/sql/t-sql/functions/stddev-transact-sql?view=sql-server-ver17>
120. STRING_SPLIT() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/string-split-transact-sql?view=sql-server-ver17>
122. SUM() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/sum-transact-sql?view=sql-server-ver17>
124. THROW - <https://learn.microsoft.com/en-us/sql/t-sql/language-elements/throw-transact-sql?view=sql-server-ver17>
126. TOP WITH TIES - <https://learn.microsoft.com/en-us/sql/t-sql/queries/top-transact-sql?view=sql-server-ver17>
128. TRUNCATE TABLE - <https://www.postgresql.org/docs/current/sql-truncate.html>
130. TRY_CAST() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/try-cast-transact-sql?view=sql-server-ver17>
132. TRY_PARSE() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/try-parse-transact-sql?view=sql-server-ver17>
134. UNIQUE - <https://learn.microsoft.com/en-us/sql/relational-databases/tables/unique-constraints-and-check-constraints?view=sql-server-ver17>
136. UPDATE with JOIN - <https://learn.microsoft.com/en-us/sql/t-sql/queries/update-transact-sql?view=sql-server-ver17>
138. VARIANCE / VAR_SAMP - <https://learn.microsoft.com/en-us/sql/t-sql/functions/var-transact-sql?view=sql-server-ver17>

139. WHERE -

<https://www.postgresql.org/docs/current/sql-select.html#SQL-WHERE>

141. WITH (NOLOCK) - <https://learn.microsoft.com/en-us/sql/t-sql/queries/hints-transact-sql-table?view=sql-server-ver17>**140. WINDOW clause -**

<https://www.postgresql.org/docs/current/sql-select.html#SQL-WINDOW>

142. XACT_STATE() - <https://learn.microsoft.com/en-us/sql/t-sql/functions/xact-state-transact-sql?view=sql-server-ver17>